

truly resembles randomness depends on mathematical definitions and theory that we'll explore soon.

Let's start by looking at a simple pseudorandom number generation algorithm and examine how much its outputs appear to resemble randomness.

## ***Linear Congruential Generators***

One of the simplest examples of a *pseudorandom number generator* (PRNG) is the *linear congruential generator* (LCG). To implement this algorithm, you'll have to choose three numbers, which we'll call  $n_1$ ,  $n_2$ , and  $n_3$ . The LCG starts with some natural number (like 1) and then simply applies the following equation to get the next number:

$$next = (previous \times n_1 + n_2) \bmod n_3$$

This is the whole algorithm, which you could say takes only one step. In Python, we'll write % instead of *mod*, and we can write a full LCG function as in [Listing 5-8](#).

---

```
def next_random(previous, n1, n2, n3):  
    the_next = (previous * n1 + n2) % n3  
    return(the_next)
```

---

### ***[Listing 5-8](#): A linear congruential generator***

Note that the `next_random()` function is deterministic, meaning that if we put the same input in, we'll always get the same output. Once again, our PRNG has to be this way because computers are always deterministic. LCGs do not generate truly random numbers, but rather numbers that look random, or are *pseudorandom*.

In order to judge this algorithm for its ability to generate pseudorandom numbers, it might help to look at many of its outputs together. Instead of getting one random number at a time, we could compile an entire list with a function that repeatedly calls the `next_random()` function we just created, as follows: